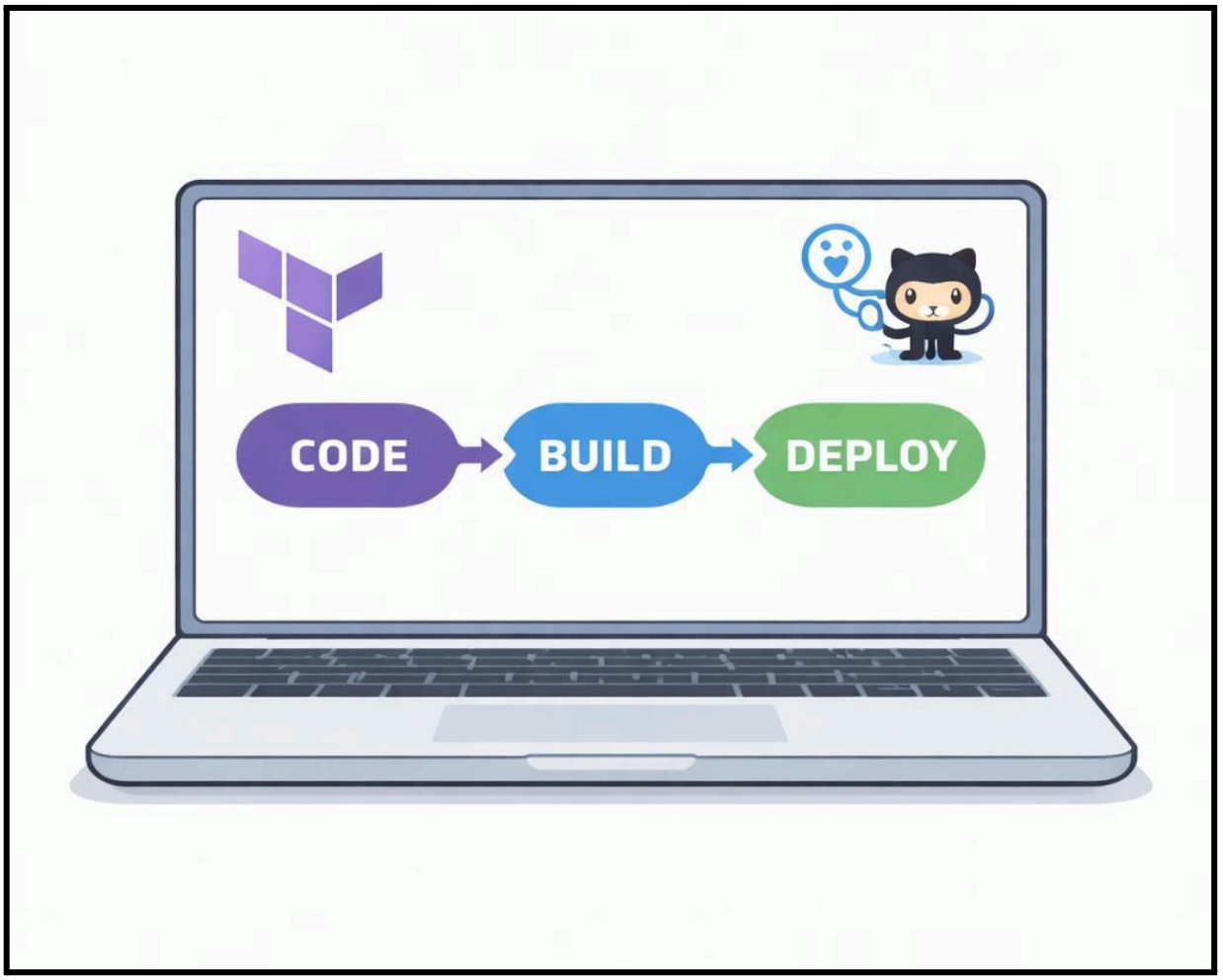
CI/CD amb Terraform i GitHub Actions



MOPT-Cloud Computing

Myrna Prieto

Frederick Vargas

Raúl Luna

Joel Sánchez

Índice

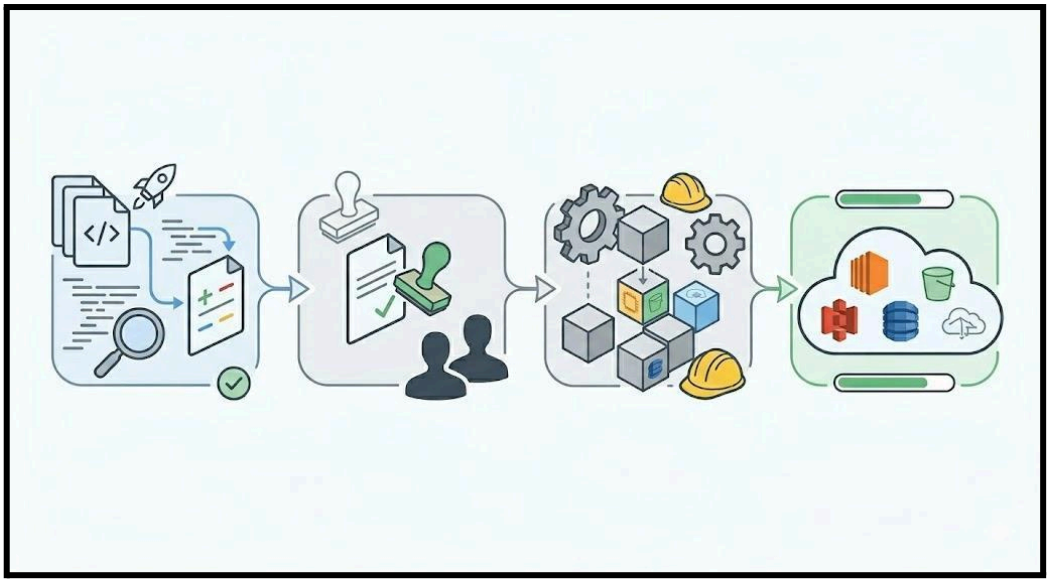
Índice....2
1. Introducción....3
2. Diagrama de flujo....4
3. Implementación del proyecto....5
3.1 Bucket S3 y DynamoDB....5
3.1.1 ¿Para qué sirve el bucket S3?....5
3.1.2 ¿Para qué sirve el DynamoDB?....5
3.1.3 ¿Cómo se ha implementado?....5
3.2 Github Secrets....6
3.2.1 ¿Cómo protegemos nuestras cuentas?....6
3.2.2 ¿Por qué hemos elegido este método?....6
3.2.3 Configuración Secretos en GitHub....6
3.3 Script CI/CD....8
3.3.1 Implementación de la fase de CI....9
3.3.2 Implementación de la fase de CD....10
3.3.3 Conceptos finales....10
3.4 EKS Clúster amb Terraform....10
3.4.1 Qué es EKS y qué se gestiona....11
AMPLIAR?....11
3.4.2 Estructura del código Terraform....11
3.4.3 Estructura de la red AWS orientada a EKS....12
4. Dificultades....12
5. Trabajo a futuro....14
6. Conclusión....15

1. Introducción

Repositorio

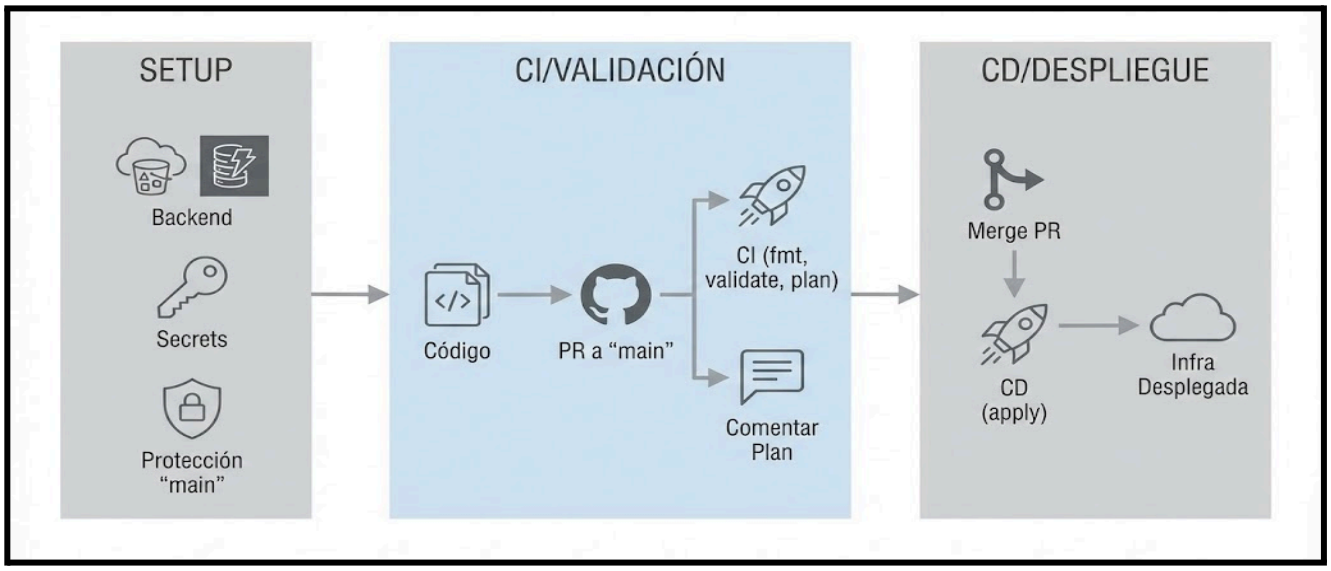
Para este proyecto utilizamos Terraform, una herramienta que nos permite crear y gestionar servicios en la nube escribiendo código en archivos en lugar de configurar todo a mano. Tambien utilizamos el Github Actions para utilizar terraform de una forma más automática, el trabajo tendrá los siguientes puntos claves:

- Revisión automática, nos referimos a que cuanto subimos nuestro código, el sistema automáticamente comprueba que no haya errores y nos enseña un borrador con los cambios que va a realizar a AWS.
- Visto Bueno, Antes de cambiar nada en AWS, una persona o varias en nuestro caso podemos revisar ese borrador y dar su aprobación. Así se evita borrar algo importante por accidente.
- Construcción final, Cuando aprobemos el cambio, el sistema construye la infraestructura automáticamente, asegurando que todo quede exactamente como lo planeamos.
- El objetivo es alcanzar un modelo de operaciones sin errores, donde la infraestructura sea consistente, escalable y está protegida contra despliegues accidentales, permitiendo al equipo centrarse en innovar y no en configurar manualmente.



2. Diagrama de flujo

Explicación visual de en qué orden se irán desarrollando los pasos de todo el workflow.



En el apartado de **SETUP** se configuran las bases para trabajar de forma segura y colaborativa. Primero hacemos un Backend S3 con un DynamoDB para que Terraform guarde el "estado" (el registro de lo que está desplegado) en la nube y no en el ordenador de un solo desarrollador. Después crearemos las Secrets de github para guardar las credenciales de AWS de forma oculta para que el pipeline pueda conectarse a la nube sin que las contraseñas queden expuestas en el código. Por último configuramos la rama main para que no se pueda subir código sin revisión previa.

En el apartado **CI/VALIDACIÓN** se basa en la automatización cada vez que alguien quiere añadir o cambiar algo. Primero tenemos un código terraform, para proponer sus cambios, crea un pull Request hacia la rama main. Al instante se ejecuta Github Actions haciendo un terraform fmt para dar formato, validate para comprobar errores de sintaxis y plan para ver que se va a crear o destruir. Por último el bot coge el resultado de terraform plan y lo escribe como un comentario directamente en el PR. Así se ven los cambios que se harán.

Por último está el apartado **CD/DESPLIEGUE** donde los cambios que se aceptan se hacen realidad en AWS. Si aprobamos el comentario del plan hace merge el PR hacia main. Cuando pasa esto Github actions ejecuta terraform apply -auto-approve. Por último Terraform se conecta a AWS y crea los recursos reales del código.

3. Implementación del proyecto

3.1 Bucket S3 y DynamoDB

3.1.1 ¿Para qué sirve el bucket S3?

El bucket S3 sirve para utilizar un archivo llamado *"terraform.tfstate"*, este sirve para recordar qué recursos ha creado en la nube. Si no tuviéramos el bucket y cada miembro ejecutará Terraform en su ordenador cada uno tendría un archivo de estado diferente, causando conflictos y duplicando recursos. Al centralizarlo en S3, garantizamos que todo el equipo y el pipeline de CI/CD consulten exactamente la misma *"fuente"* de estado antes de aplicar cualquier cambio. Además, hemos activado el versionado del bucket para poder recuperar estados anteriores en caso de desastre.

3.1.2 ¿Para qué sirve el DynamoDB?

DynamoDB se utiliza para evitar el riesgo de que el archivo de estado se corrompa debido a ejecuciones simultáneas por ejemplo, si dos personas aprueban y fusionan un Pull Request al mismo tiempo. La solución a esto es la gestión del bloqueo de estado (State Locking). Cuando Github Actions o un usuario empieza a ejecutar un *"terraform apply"*, terraform crear un registro (LockID) en esta tabla, si otro proceso intenta ejecutarse a la vez, el dynamodb se lo impide hasta que el primero termine, evitando que el archivo de estado se corrompa.

3.1.3 ¿Cómo se ha implementado?

En lugar de crear el Bucket S3 y la DynamoDB manualmente desde la consola AWS, hemos hecho una infraestructura como código usando terraform y subido a una carpeta separada llamada *"bootstrap"* dentro de nuestro repositorio. Lo hacemos así para que no estorbe en el código principal ni de error y también para mostrar al profesorado como hemos hecho el Bucket y el DynamoDB.

Si quisiéramos desplegar el proyecto desde cero, primero iríamos a la carpeta *"bootstrap"* y ejecutamos un "terraform init" y "terraform apply" localmente ya que es el único requisito que necesitamos para empezar el proyecto.

3.2 Github Secrets

Para que nuestro flujo de trabajo sea seguro y profesional, hemos decidido no dejar rastro de nuestras claves de acceso en el código. Para ello, hemos implementado el uso de GitHub Actions Secrets.

3.2.1 ¿Cómo protegemos nuestras cuentas?

En lugar de escribir las llaves de AWS dentro de los archivos de Terraform (lo cual sería un peligro si el repositorio es público o compartido), las hemos guardado en una "zona segura" dentro de GitHub.

3.2.2 ¿Por qué hemos elegido este método?

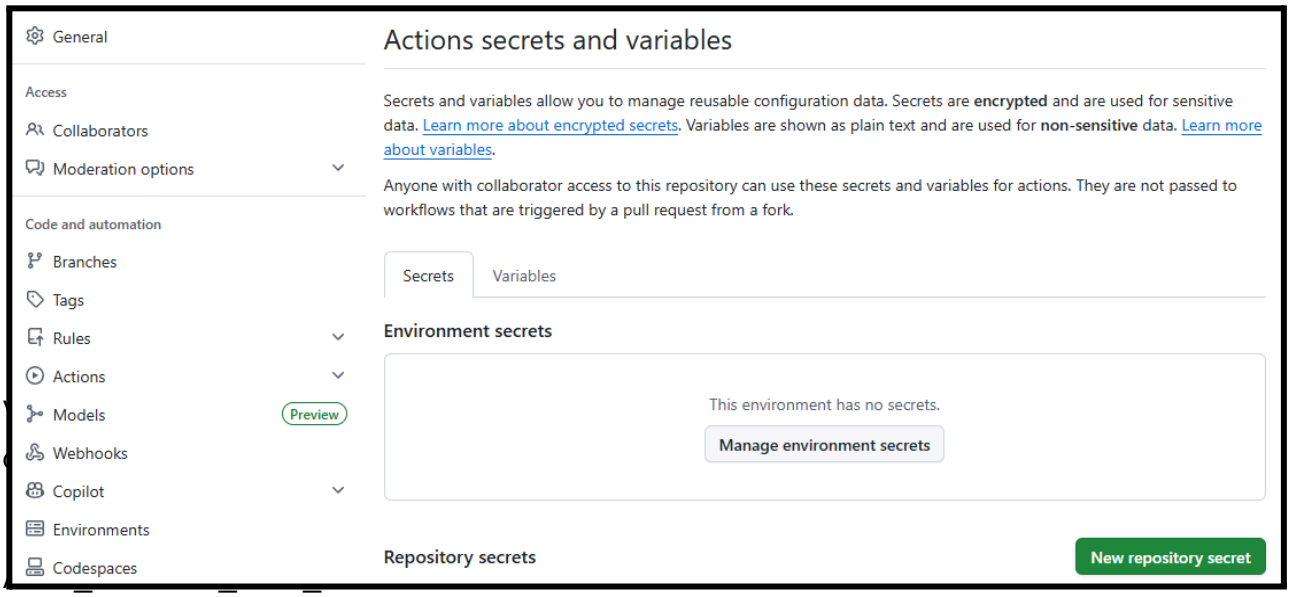
Hemos usado este sistema por dos razones principales:

-Seguridad: Nadie del grupo, ni nadie de fuera, puede ver las claves una vez guardadas en GitHub. Solo las máquinas que ejecutan el código pueden usarlas.

-Sincronización: Al estar las claves en el repositorio común, cualquiera de nosotros puede lanzar un cambio (vía Pull Request) y el sistema podrá desplegarse automáticamente sin que tengamos que pasarnos las claves por WhatsApp o email.

3.2.3 Configuración Secretos en GitHub

Entramos en Settings > Secrets and variables y le damos a “New repository secret”.



AWS_SECRET_ACCESS_KEY=?????

AWS_SESSION_TOKEN=?????

AWS CLI:
Copy and paste the following into ~/.aws/credentials

```
[default]
aws_access_key_id=ASIA4TT5EDZCLEDQTV3J
aws_secret_access_key=sUng4gnLIdSyXHqTnC7GaaeNXOyX18qb7uUone5a
aws_session_token=IQoJb3JpZ21uX2VjEKD/////////wEaCXVzLXdlc3QtMiJI
MEYCIQCkGgGSCIh8QrK7btKgJG3qzhGApEzdtIKStsoRvJ4ZrAIhAPnLM0BZFLJzZU
```

En new repository secret añadimos las claves a cada uno

Actions secrets / Update secret

AWS_ACCESS_KEY_ID

Value

ASIA4TT5EDZCAL7VEPQC

Aquí tenemos los tres secretos creados con sus respectivas claves y no podemos ver la clave porque se ocultan.

Repository secrets

New repository secret

Name	Last updated
AWS_ACCESS_KEY_ID	1 hour ago
AWS_SECRET_ACCESS_KEY	1 hour ago
AWS_SESSION_TOKEN	1 hour ago

3.3 Script CI/CD

Esto es lo que podríamos llamar el “core” del trabajo. La base de la automatización de nuestras tareas.

La Integración Continua (CI) es una práctica de desarrollo que consiste en integrar de forma frecuente los cambios realizados en el código dentro de un repositorio común, activando automáticamente procesos de validación. El objetivo principal es reducir fallos en etapas posteriores, asegurar que el código es consistente y mantener el repositorio siempre en un estado estable. El Despliegue Continuo (CD) es la práctica complementaria que automatiza la entrega de los cambios una vez han superado las validaciones de la CI. El objetivo es eliminar tareas manuales repetitivas, reducir errores humanos y garantizar que los cambios aprobados se desplieguen de forma consistente y reproducible.

La automatización se ha realizado mediante GitHub Actions, definiendo un workflow en formato YAML dentro del directorio *.github/workflows/.* Este archivo es el que describe cuándo se ejecuta el pipeline, en qué entorno, qué permisos necesita y qué pasos debe seguir para validar y desplegar la infraestructura definida en Terraform.

Para garantizar que el flujo funcione correctamente y no pudiera ser evitado, se configuró la protección de la rama main desde la sección *Settings → Branches* del repositorio. Se activó la opción de “*Require a pull request before merging*” y se exigió al menos una aprobación antes de permitir el merge. De esta forma se impide hacer *push* directo a main, obligando a que todos los cambios pasen por una Pull Request y, por tanto, por la ejecución automática del terraform plan.

El workflow se ha definido con dos eventos principales en el bloque on:

- *pull_request* sobre la rama main (CI)
- *push* sobre la rama main (CD)

Además, se ha configurado el bloque permissions para permitir: *contents: read, pull-requests: write y issues: write*

Esto es importante porque el job de CI publica automáticamente el resultado del terraform plan como comentario en la PR, y para ello necesita permisos de escritura sobre pull requests.

3.3.1 Implementación de la fase de CI

La fase de CI se ejecuta únicamente cuando el evento es *pull_request*. Cada job de GitHub Actions se ejecuta en una máquina virtual efímera (en este caso ubuntu-latest). Eso significa que no hay estado compartido entre ejecuciones y que todo debe configurarse desde cero en cada ejecución.

El job sigue esta secuencia:

1. Checkout del repositorio: Este paso clona el repositorio en la máquina virtual temporal. Sin este paso, no existirían los archivos .tf en el entorno de ejecución.

2. Configuración de Terraform: Este paso instala la versión específica de Terraform indicada. Es importante fijar la versión para evitar comportamientos distintos entre entornos locales y el pipeline.

3. Configuración de credenciales AWS: Este paso exporta variables de entorno AWS dentro del runner. Terraform utiliza estas credenciales para inicializar el backend remoto (S3), leer el estado y ejecutar el plan contra AWS. Es importante entender que cada job necesita configurar sus propias credenciales, porque los jobs no comparten entorno entre sí.

4. Inicialización de Terraform: Este paso descarga el provider de AWS, inicializa el backend remoto y prepara el entorno para validate y plan. Sin este paso, terraform validate puede fallar por falta de provider.

5. Validación: Se comprueba la sintaxis, se verifica coherencia de bloques y se detectan referencias inválidas. No interactúa con AWS todavía, pero sí requiere que el provider esté descargado.

6. Generación del plan: El resultado del plan se redirige a un archivo (plan_logs.txt) en lugar de mostrarlo solo por consola. Esto permite reutilizar este contenido.

7. Publicación automática del plan en la PR: Usando el script se lee el archivo plan_logs.txt, construye un bloque Markdown, y llama a la API de GitHub para crear un comentario en la PR.

3.3.2 Implementación de la fase de CD

La fase CD se ejecuta cuando ocurre un push a main. Este push proviene de una PR que ha sido aceptada. La estructura del job es similar al CI, pero más simplificada.

1. Checkout y setup de Terraform: Se repiten los pasos de actions/checkout, hashicorp/setup-terraform, y configuración de credenciales AWS. Es obligatorio repetirlos porque el runner es nuevo y no conserva nada del job anterior.
2. Inicialización: Se vuelve a inicializar el backend remoto y providers con terraform init.
3. Apply automático: En este punto final Terraform compara estado remoto y configuración, aplica los cambios, actualiza el terraform.tfstate en S3 y configura el locking.

3.3.3 Conceptos finales

Más allá de acelerar procesos, la implantación de CI/CD introduce un cambio estructural en la forma de gestionar la infraestructura: desplaza el control desde intervenciones manuales hacia un sistema verificable, trazable y determinista. Esto no solo reduce errores operativos, sino que convierte cada modificación en un evento observable y validado antes de impactar en producción. En consecuencia, la automatización deja de ser únicamente una mejora de eficiencia para convertirse en un mecanismo de gobernanza técnica que refuerza la estabilidad, la transparencia y la confianza en la evolución del sistema.

3.4 EKS Clúster amb Terraform

Esta parte del proyecto se encarga de aprovisionar de manera declarativa y reproducible un clúster de Kubernetes en AWS gestionado mediante Amazon EKS (Elastic Kubernetes Service), dejándolo listo para ejecutar pods de manera escalable.

El diseño del proyecto separa la infraestructura subyacente de los objetos de Kubernetes. Terraform crea la infraestructura base en la nube de AWS (mediante los archivos contenidos en "./terraform/," y los manifiestos de la aplicación de ejemplo "./demo-app/" se aplican a posteriori al clúster existente.

3.4.1 Qué es EKS y qué se gestiona

Kubernetes se divide lógicamente en dos grandes bloques: el Control Plane y los Worker Nodes

- Control Plane: Es el que expone la API de Kubernetes, almacena el estado de todos los objetos en la base de datos de clave valor (etcd) y ejecuta los controladores que son responsables de reconciliar el estado deseado con el estado real. Por ejemplo, si declaramos que queremos 3 réplicas de nuestra aplicación, el controlador evalúa el estado actual y se asegura de levantar los pods necesarios hasta alcanzar este número.
- Workers: Son las máquinas virtuales donde realmente corren y se ejecutan nuestros contenedores (agrupados en Pods).

Normalmente, desplegar y mantener el Control Plane es una tarea bastante compleja. Con Amazon EKS, AWS ofrece el control plane como un servicio gestionado. AWS se encarga de aprovisionar, garantizar su alta disponibilidad desplegándolo a través de varias AZs, entre muchas otras ventajas.

Por otro lado, nosotros, como arquitectos de la infraestructura, mediante Terraform, nos encargamos de gestionar el entorno que rodea al control plane: la red, los grupos de seguridad, los roles … Y de instanciar la flota de Worker Nodes que se conectarán al servicio EKS para ejecutar las cargas de trabajo bajo nuestra gestión.

3.4.2 Estructura del código Terraform

El código Terraform sigue las prácticas habituales de modularidad, dividiendo las responsabilidades en diferentes archivos dentro del directorio "./terraform/":

- main.tf: Es el núcleo del aprovisionamiento. Contiene la declaración de todos los recursos de red, los grupos de seguridad, y la configuración tanto del clúster de EKS como de su grupo de nodos.

- variables.tf: Define los parámetros de entrada del proyecto, permitiendo que la infraestructura sea parametrizable. Podemos definir por ejemplo el rango de la VPC o las variables de autoescalado como *"node_desired_size"* y *"node_max_size"*.

- outputs.tf: Muestra información clave una vez finalizado el despliegue, como puede ser el nombre del clúster o el endpoint de la API, útil para conectar otros servicios.

- providers.tf: Configura el proveedor de AWS, indicando la región y las versiones aceptadas/esperadas tanto de Terraform como de AWS.

- backend.tf: Configura el almacenamiento remoto del estado de Terraform en un bucket S3 protegido con bloqueos concurrentes mediante DynamoDB, permitiendo el trabajo seguro en equipo y en pipelines de CI/CD.

Uno de los motivos principales de utilizar Terraform es la reproducibilidad y trazabilidad. Al definir IaaC, podemos levantar o destruir todo el entorno idéntico con un solo comando. Esto es fundamental para ahorrar costes en entornos de desarrollo (como es nuestro caso con AWS Academy).

3.4.3 Estructura de la red AWS orientada a EKS

La topología de red es la siguiente:

- VPC: Creamos una red privada virtual en la región configurada con un bloque CIDR amplio (10.0.0.0/16). Esta VPC actúa como el contenedor de red para el resto de la infraestructura.
- Subnets Públicas y privadas: El espacio de direcciones se divide en subredes distribuidas simétricamente a lo largo de 2 Zonas de Disponibilidad para garantizar tolerancia a fallos.
    - Subnets Públicas: Tienen salida bidireccional a Internet mediante un Internet Gateway (IGW). Estas subredes están etiquetadas específicamente para que EKS detecte automáticamente que ahí es donde debe aprovisionar balanceadores de carga accesibles desde el exterior.
    - Subnets Privadas: No disponen de salida directa. Se rutean a través de un NAT Gateway ubicado en la subred pública, de forma que puedan descargar imágenes de contenedores (pull) sin exponer sus servicios internos. Siguiendo el principio de mínimo privilegio, el Node Group de EKS se despliega íntegramente en estas subredes privadas aisladas.

3.4.4 Aplicación de demostración

Para validar el correcto funcionamiento de la topología de red y el clúster perimetral, se ha desarrollado una aplicación web didáctica en Python (usando Flask) que se empaqueta en una imagen de Docker.

El objetivo de la aplicación es demostrar visualmente el balanceo de carga nativo orquestado por EKS. A nivel lógico, la aplicación:

1. Captura el Hostname del Pod y su dirección IP interna en cada petición.
2. Genera una interfaz dinámica donde el color de fondo cambia algorítmicamente en función del nombre del Pod subyacente que está respondiendo.

Para materializar este despliegue en Kubernetes se ha configurado un manifiesto "./demo-app/k8s-manifest.yaml" que provisiona dos recursos clave:

- Un Deployment, configurado para asegurar la ejecución de 3 réplicas distribuidas de la aplicación, controlando aspectos como los límites de memoria y salud (Readiness Probes) de los contenedores.

- Un Service de tipo LoadBalancer. Gracias a la integración nativa de EKS con AWS, añadiendo una simple anotación *<<service.beta.kubernetes.io/aws-load-balancer-type: "nlb">>*, el Control Plane solicita automáticamente a AWS la creación de un verdadero Network Load Balancer. Este NLB ejerce de punto de entrada redirigiendo el tráfico HTTP de forma equilibrada hacia los contenedores aislados en las subredes privadas.

Al recargar la página web del balanceador, vemos cómo va cambiando la dirección IP y el nombre del Pod. Esto demuestra de forma práctica que Kubernetes está funcionando bien, coordinando la respuesta y repartiendo el trabajo entre los diferentes nodos del clúster EKS.

4. Dificultades

Las dificultades en este proyecto han sido mayoritariamente causadas por la sincronización del equipo. Ya que aunque nos hemos dividido en trabajo en diferentes partes, a la hora de avanzar ciertas cosas o hacer pruebas es necesario del input en directo de los compañeros. Quitando eso, un fallo grande que hemos tenido es pensar que el comando *terraform fmt* funciona igual en la terminal que en las actions. En terminal arreglaba y formateaba el código, sin embargo en las acciones solo valida que esté correctamente formateado, por lo que este paso hay que hacerlo antes del commit.

Ha habido también problemas con la sincronización de las ramas, ya que al empezar el proyecto no teníamos una estructura demasiado clara y esto nos perjudicó según íbamos haciendo. Nuestro conocimiento básico de git hizo que cometieron errores que crearon muchos conflictos que podrían haber sido prevenidos.

Otro tema ha sido el de los permisos y las protecciones de las ramas, ya que al inicialmente crear un repositorio personal los permisos no eran modificables. Tuvimos que migrar el proyecto a un repositorio de una organización para ganar más control en este terreno.

En la parte de los Secrets el único “problema” que ha surgido ha sido el tema de ir actualizando las claves secretas que nos da el AWS Details del Learner Lab.

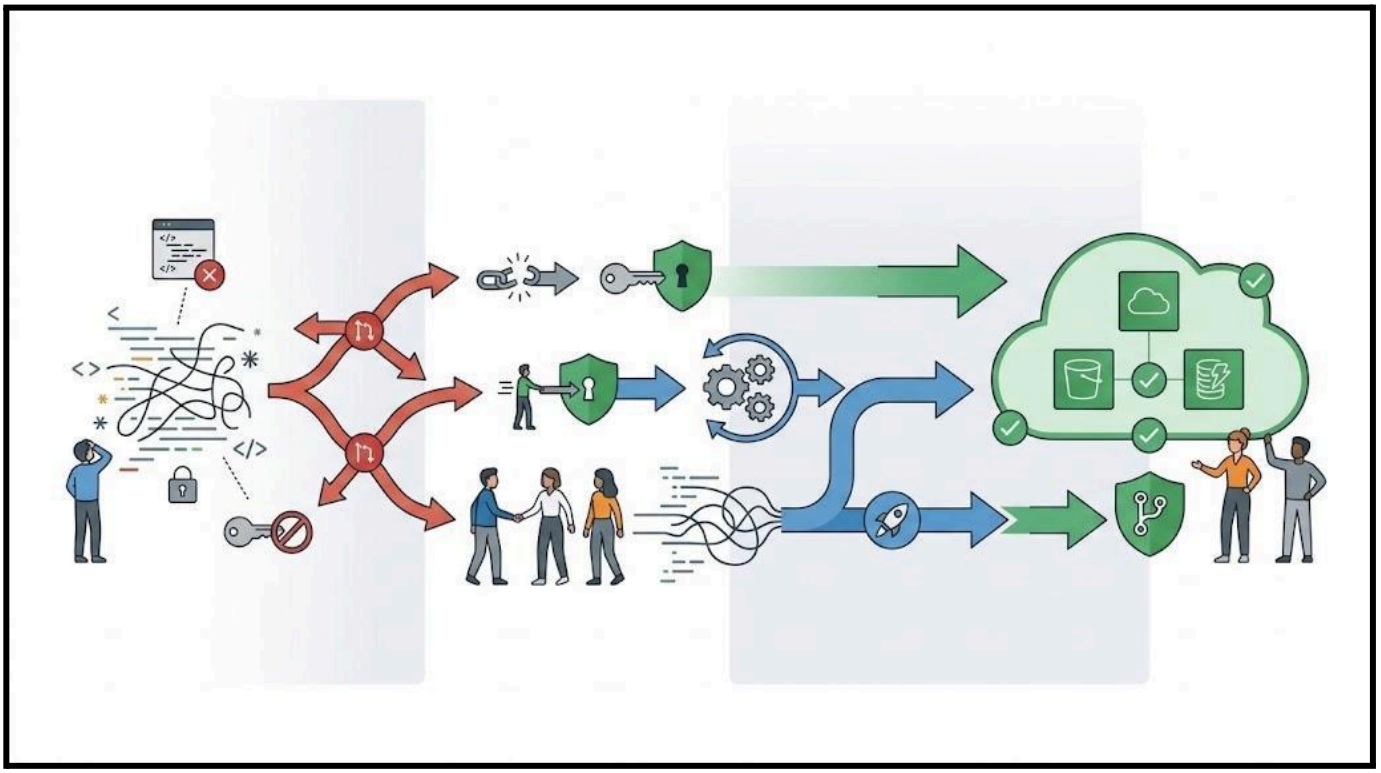
5. Trabajo a futuro

Nos planteamos en reforzar la estructura del pipeline separando claramente la parte del CI y del CD, ya sea en archivos distintos o en diferentes jobs. También queremos asegurarnos de que ambos tengan las credenciales y el directorio de trabajo bien configurados de forma consistente. Otra idea muy práctica es añadir una validación rápida de formato (terraform fmt -check) justo antes de validar o planificar, para que el sistema falle rápido y nos avise al instante si hay errores, dándonos un feedback mucho más directo.

Sobre el flujo de las Pull Requests, queremos dejar las reglas del juego mucho más claras y documentadas. La idea es definir bien quién puede subir código a las ramas principales, quién tiene permisos para aprobar, y configurar el repositorio para que nadie pueda auto-aprobarse sus propios cambios. Como un extra más avanzado para el futuro, nos gustaría añadir una regla que bloquee automáticamente la fusión si el plan de Terraform detecta que se van a destruir recursos (por ejemplo, si el sistema lee un "Plan: X to destroy").

Por último, necesitamos acordar dentro del equipo una forma estándar de trabajar con Git (decidir y configurar si preferimos usar rebase o merge) para reducir al máximo los conflictos y evitar que las ramas se descontrolen.

6. Conclusión

Este proyecto nos ha servido para montar un flujo real de CI/CD con Terraform y GitHub Actions, logrando que la infraestructura se valide y despliegue automáticamente de forma segura. Ha sido un proceso bastante duro, sobre todo al principio por la falta de experiencia del equipo trabajando juntos con Git; tuvimos bastantes conflictos con las ramas y problemas de sincronización. Además, nos encontramos con imprevistos como tener que migrar todo a una "Organización" en GitHub para poder configurar bien los permisos, o lidiar con las credenciales temporales de AWS y el comportamiento de comandos como terraform fmt dentro del pipeline.

Sin embargo, precisamente esos tropiezos son los que más nos han enseñado. Aunque nos ha costado coordinarnos y superar estos bloqueos técnicos, el esfuerzo ha valido la pena. Terminamos el proyecto no solo con un despliegue que funciona perfectamente y está protegido frente a errores, sino con una idea mucho más clara y práctica de lo que significa trabajar en equipo con infraestructura como código en el mundo real.